Task Manager

ระบบจัดการงานส่วนตัวแบบลิสต์รายการ

รายงาน Mini Project: สร้างเว็บไซต์แบบ Full-Stack

ด้วย REST API Node.js + Express.js + HTML / CSS / JavaScript

จัดทำโดย

นายจตุชัย เดชสูงเนิน รหัสนักศึกษา 660910658

รายวิชา SELECTED TOPICS IN ELECTRONICS AND COMPUTER ENGINEERING I

GitHub Repository: https://github.com/Mosfox/miniProject

สารบัญ

1. บทนำ
2. เครื่องมือและเทคโนโลยีที่ใช้
3. โครงสร้างโปรเจกต์
4. การออกแบบ Resource และ REST API
5. การพัฒนาฝั่งเซิร์ฟเวอร์ (Backend)
6. การพัฒนาฝั่งไคลเอนต์ (Frontend)
7. การทดสอบและภาพหน้าจอการทำงาน
8. ปัญหาที่ต้องระวังและวิธีจัดการ
9. สรุปผลและแนวทางพัฒนาต่อ

1. บทนำ

1.1 ที่มาและเหตุผลที่เลือกหัวข้อ

โจทย์ Mini Project นี้ให้เลือก resource ของแอปเอง 1 หัวข้อ เราเลือก Task Manager หรือระบบจัดการงานส่วนตัว เพราะเป็นสิ่งที่ตัวเองใช้จริงทุกวัน ทั้งงานส่ง อาจารย์ งานบ้าน และเรื่องส่วนตัวอื่น ๆ ทำให้คิดฟีเจอร์ได้ง่ายว่าควรมีอะไร อีกเหตุผลคือ resource ตัวนี้มีข้อมูลไม่ซับซ้อน (ชื่องาน หมวดหมู่ ความสำคัญ สถานะเสร็จ/ไม่เสร็จ) จึงเหมาะกับการโฟกัสที่การเรียนรู้ REST API ให้ครบ CRUD มากกว่าไปยุ่งกับโครงสร้างข้อมูลที่ยาก

1.2 วัตถุประสงค์

- ฝึกสร้าง REST API ด้วย Node.js และ Express.js ให้ครบทั้ง 5 เส้นทางของ CRUD
- ฝึกใช้ HTTP method และ status code ให้ถูกความหมาย (200, 201, 204, 400, 404)
- ฝึกเขียนหน้าเว็บที่เรียก API ด้วย `fetch()` และอัปเดตหน้าจอโดยไม่ต้องโหลดหน้าใหม่
- ฝึกจัดโครงสร้างโปรเจกต์ เขียน README และใช้ Git/GitHub ส่งงาน

1.3 ขอบเขตและข้อกำหนดของงาน

ตามที่โจทย์กำหนด งานนี้ต้องมีองค์ประกอบดังนี้ เราใช้เป็นเช็กลิสต์ระหว่างทำงาน:

ข้อกำหนด	สิ่งที่ทำในงานนี้
CRUD ครบผ่าน REST API	มี 5 endpoint ที่ `/api/tasks` (ดูหัวข้อ 4)
GET รายการทั้งหมด กรองด้วย query string อย่างน้อย 1 ตัว	กรองได้ 3 ตัว: `done`, `category`, `priority`
GET ตาม id ตอบ 404 ถ้าไม่พบ	ทำแล้ว
POST ตรวจข้อมูล ตอบ 400 / 201	ตรวจ title, priority, done
PATCH หรือ PUT ตอบ 404 ถ้าไม่พบ	เลือกใช้ PATCH
DELETE ตอบ 204	ทำแล้ว
ฝั่งไคลเอนต์ใช้ fetch(), ฟอร์มเพิ่ม, ปุ่มลบ/แก้ไข, CSS สวยงาม	ดูหัวข้อ 6
เรียก API ของตัวเองเท่านั้น	ทุก `fetch()` ชี้ไป ที่ `/api/tasks` เท่านั้น
README, รายงาน PDF, GitHub repo เดียว	อยู่ใน repository นี้ (โฟลเดอร์ `docs/`)

2. เครื่องมือและเทคโนโลยีที่ใช้

2.1 ฝั่งเซิร์ฟเวอร์: Node.js และ Express.js

Node.js ทำให้เขียน JavaScript ฝั่งเซิร์ฟเวอร์ได้ ส่วน Express.js เป็นเฟรมเวิร์กที่ช่วยให้กำหนด route และจัดการ request/response ได้สั้นและอ่านง่าย
ตัวอย่างเช่น `app.get('/api/tasks', ...)`บรรทัดเดียวก็ได้ endpoint แล้ว งานนี้ใช้ Express เวอร์ชัน 4 และไม่ต้องติดตั้งไลบรารีอื่นเพิ่ม

2.2 ฝั่งไคลเอนต์: HTML, CSS, JavaScript

หน้าเว็บเขียนด้วย HTML/CSS/JavaScript ล้วน ไม่ใช้เฟรมเวิร์ก เพื่อให้เห็นชัดว่า `fetch()` คุยกับ API อย่างไร การสร้างองค์ประกอบในหน้าใช้ DOM API (`createElement`, `textContent`) แทนการต่อสตริง HTML

2.3 เครื่องมือช่วยพัฒนาและทดสอบ

- npm: ติดตั้งแพ็กเกจ และรันสคริปต์ `npm run dev` (ใช้ `node --watch` ให้เซิร์ฟเวอร์รีสตาร์ทเองเมื่อแก้โค้ด)
- Browser DevTools (แท็บ Network และ Console): ดู request/response และ status code จริง
- curl หรือ Postman: ยิง API ตรง ๆ เพื่อทดสอบกรณีที่หน้าเว็บไม่ได้ส่ง เช่น ส่งข้อมูลไม่ครบ
- Git และ GitHub: เก็บประวัติการแก้ไขและส่งงาน

3. โครงสร้างโปรเจกต์

3.1 โครงสร้างโฟลเดอร์

```
task-manager/
>server/
  > index.js       # เซิร์ฟเวอร์ Express และ REST API
>public/
  > index.html     # โครงหน้าเว็บ
  > style.css      # การตกแต่ง
  > app.js         # โค้ดฝั่งไคลเอนต์ (fetch)
>docs/             # รายงาน PDF และภาพ screenshot
>package.json
> README.md
```

3.2 หน้าที่ของแต่ละไฟล์

- `server/index.js`: สร้างเซิร์ฟเวอร์ กำหนด route ทั้งหมด เก็บข้อมูล และตรวจสอบข้อมูลที่ส่งเข้ามา
- `public/index.html`: มีฟอร์มเพิ่มงาน แท็บกรอง และที่ว่างสำหรับแสดงรายการ
- `public/app.js`: เรียก API แล้ววาดรายการงานลงหน้าเว็บ จัดการปุ่มต่าง ๆ
- `public/style.css`: ธีมสี ระยะห่าง และ responsive

3.3 เหตุผลที่แยก /server กับ /public

ถ้าเอาโค้ดสองฝั่งไปรวมกันจะสับสนว่าไฟล์ไหนรันที่เซิร์ฟเวอร์ ไฟล์ไหนรันที่เบราว์เซอร์ การแยกโฟลเดอร์ทำให้ชัดเจนว่า `/public` คือไฟล์ที่ส่งให้ผู้ใช้ (เซิร์ฟเวอร์เปิดให้ด้วย `express.static`) ส่วน `/server`คือโค้ดที่ผู้ใช้มองไม่เห็น นอกจากนี้ยังทำให้อนาคตแยกไปทำเป็นสองโปรเจกต์ได้ง่ายขึ้น

4. การออกแบบ Resource และ REST API

4.1 โครงสร้างข้อมูลของ Task

ฟิลด์	ชนิด	คำอธิบาย	บังคับ?
`id`	number	รหัสงาน เซิร์ฟเวอร์สร้างให้เอง เพิ่มทีละ 1	สร้างอัตโนมัติ
`title`	string	ชื่องาน	ใช่
`category`	string	หมวดหมู่ เช่น เรียน, ส่วนตัว ถ้าไม่ใส่จะเป็น "ทั่วไป"	ไม่
`priority`	string	`low` / `medium` / `high` ค่าเริ่มต้น `medium`	ไม่
`done`	boolean	ทำเสร็จแล้วหรือยัง ค่าเริ่มต้น `false`	ไม่
`createdAt`	string	เวลาที่สร้าง (ISO 8601) เซิร์ฟเวอร์ใส่ให้	สร้างอัตโนมัติ

```
{
  "id": 1,
  "title": "ส่งรายงานวิชา Web Development",
  "category": "เรียน",
  "priority": "high",
  "done": false,
  "createdAt": "2026-09-30T08:00:00.000Z"
}
```

4.2 สรุป Endpoint ทั้งหมด

Method	Endpoint	หน้าที่	Status ที่ตอบ
GET	`/api/tasks`	ดึงรายการทั้งหมด (กรองได้)	200
GET	`/api/tasks/:id`	ดึงงานเดียว	200, 404
POST	`/api/tasks`	เพิ่มงานใหม่	201, 400
PATCH	`/api/tasks/:id`	แก้ไขบางฟิลด์	200, 400, 404
DELETE	`/api/tasks/:id`	ลบงาน	204, 404

ตั้งชื่อ path เป็นคำนามพหูพจน์ (`tasks`) และให้ HTTP method เป็นตัวบอกว่าทำอะไร ไม่ใส่คำกริยาใน URL เช่น ไม่ใช้ `/api/addTask`

4.3 GET /api/tasks: ดึงรายการและกรองด้วย Query String

ถ้าไม่ส่งพารามิเตอร์จะได้งานทั้งหมด ถ้าส่งจะกรองเพิ่มทีละเงื่อนไข และใช้ร่วมกันได้

พารามิเตอร์	ตัวอย่าง	ความหมาย
`done`	`?done=true`	เฉพาะงานที่เสร็จแล้ว (`false` = ที่ยังค้าง)
`category`	`?category=เรียน`	เฉพาะหมวดหมู่นั้น
`priority`	`?priority=high`	เฉพาะระดับความสำคัญนั้น

ตัวอย่าง: GET /api/tasks?done=false&priority=high จะได้งานด่วนที่ยังไม่เสร็จ ถ้าไม่มีงานตรงเงื่อนไขก็ตอบ 200 พร้อมอาร์เรย์ว่าง [] ไม่ถือเป็น error

4.4 GET /api/tasks/:id: ดึงงานเดียวด้วย Route Parameter

ค่า :id ใน URL เป็นสตริงเสมอ จึงต้องแปลงเป็นตัวเลขก่อนเทียบกับ id ของงาน ถ้าหาไม่เจอ (รวมถึงกรณีที่ส่งมาไม่ใช่ตัวเลข) จะตอบ 404 พร้อมข้อความ {"error": "ไม่พบงานที่ระบุ"}

4.5 POST /api/tasks: เพิ่มงานพร้อมตรวจสอบข้อมูล

ข้อมูลที่ส่งมาต้องผ่านการตรวจก่อนบันทึก โดยตรวจว่า

- title ต้องเป็นสตริงและไม่ว่าง (ตัดช่องว่างหัวท้ายแล้ว)
- priority ถ้าส่งมา ต้องเป็น low, medium หรือ high
- done ถ้าส่งมา ต้องเป็น boolean

ถ้าไม่ผ่านตอบ 400 พร้อมข้อความบอกสาเหตุ ถ้าผ่านจะสร้าง id และ createdAt ให้แล้วตอบ 201พร้อมข้อมูลงานที่สร้างเสร็จ เพื่อให้ฝั่งไคลเอนต์นำไปใช้ต่อได้ทันที

POST /api/tasks

{ "title": "อ่านหนังสือสอบ", "category": "เรียน", "priority": "high" }

201 Created

{ "id": 4, "title": "อ่านหนังสือสอบ", "category": "เรียน", "priority": "high", "done": false, ... }

4.6 PATCH /api/tasks/:id: แก้ไขงาน

เลือกใช้ PATCH แทน PUT เพราะเราต้องการแก้แค่บางฟิลด์ เช่น ติ๊กว่าเสร็จแล้ว ก็ส่งแค่ `{"done": true}` ไม่ต้องส่งข้อมูลทั้งก้อนใหม่ ถ้าใช้ PUT ต้องส่งครบทุกฟิลด์ทุกครั้ง ซึ่งไม่สะดวกกับปุ่มติ๊กเสร็จ

- ไม่พบ id: ตอบ 404
- ข้อมูลที่ส่งมาไม่ถูกต้อง (เช่น `title` เป็นสตริงว่าง หรือ `priority` ผิดค่า): ตอบ 400
- สำเร็จ: ตอบ 200 พร้อมงานที่แก้แล้ว

4.7 DELETE /api/tasks/:id: ลบงาน

ถ้าลบสำเร็จตอบ 204 No Content ซึ่งไม่มี body ฝั่งไคลเอนต์จึงต้องระวังไม่เรียก `res.json()` กับ response แบบนี้ ถ้าไม่พบ id ตอบ 404

4.8 สรุป Status Code และรูปแบบ Error

Status	ใช้เมื่อ
200 OK	ดึงหรือแก้ไขสำเร็จ
201 Created	เพิ่มงานสำเร็จ
204 No Content	ลบสำเร็จ
400 Bad Request	ข้อมูลไม่ครบ ไม่ถูกต้อง หรือ JSON เสียรูปแบบ
404 Not Found	ไม่พบงานตาม id

ทุก error ตอบเป็น JSON รูปแบบเดียวกันคือ `{"error": "ข้อความ"}` ทำให้ฝั่งไคลเอนต์อ่านข้อความไปแสดงผู้ใช้ได้เลยโดยไม่ต้องเขียนหลายแบบ

5. การพัฒนาฝั่งเซิร์ฟเวอร์ (Backend)

5.1 การตั้งค่า Express และ Middleware

ใช้ middleware 2 ตัวหลัก คือ express.json() เพื่ออ่าน JSON จาก body ของ request (ถ้าไม่ใส่ req.body จะเป็น undefined) และ express.static() เพื่อให้เซิร์ฟเวอร์ส่งไฟล์ในโฟลเดอร์ publicให้เบราว์เซอร์ ทำให้เปิด http://localhost:3000 แล้วเห็นหน้าเว็บทันทีโดยไม่ต้องมีเซิร์ฟเวอร์แยก

```
app.use(express.json());
app.use(express.static(path.join(__dirname, '..', 'public')));
```

5.2 การเก็บข้อมูล

เก็บข้อมูลเป็นอาร์เรย์ tasks ในหน่วยความจำ พร้อมตัวนับ nextId โจทย์ไม่ได้ให้ใช้ฐานข้อมูล และวิธีนี้ทำให้โฟกัสที่ตัว API ได้เต็มที่ ข้อเสียคือข้อมูลหายเมื่อรีสตาร์ทเซิร์ฟเวอร์ (พูดถึงอีกครั้งในหัวข้อ 9.2) เราใส่ข้อมูลตัวอย่างไว้ 3 รายการเพื่อให้เปิดมาแล้วมีของให้เห็นเลย

5.3 ฟังก์ชันตรวจสอบข้อมูล (Validation)

เขียนฟังก์ชัน validate() ไว้ที่เดียวแล้วเรียกใช้ทั้งใน POST และ PATCH โดยมีตัวเลือก partialสำหรับ PATCH ที่ไม่จำเป็นต้องส่งครบทุกฟิลด์ ฟังก์ชันคืนข้อความ error หรือ null ถ้าผ่าน ทำให้ route ไม่ยาวและไม่ต้องเขียนเงื่อนไขซ้ำ

5.4 การจัดการ Error

เพิ่ม error-handling middleware ท้ายไฟล์ เพื่อดักกรณีที่ผู้ใช้ส่ง JSON ที่รูปแบบผิดมา (เช่น วงเล็บไม่ปิด) ซึ่งถ้าไม่ดักไว้ Express จะตอบเป็นหน้า HTML ของ error มาให้ ทำให้ฝั่งไคลเอนต์อ่านไม่ได้ เราจึงแปลงเป็น 400 พร้อม JSON แทน

6. การพัฒนาฝั่งไคลเอนต์ (Frontend)

6.1 การแสดงรายการด้วย fetch() แบบ GET

เมื่อเปิดหน้า ฟังก์ชัน `loadTasks()` จะเรียก `fetch('/api/tasks')` รับ JSON แล้ววาดแต่ละงานเป็น `<li>` ลงในหน้า ทั้งหมดนี้ไม่ต้องโหลดหน้าใหม่ ถ้ารายการว่างจะแสดงข้อความชวนให้เพิ่มงานแรก แทนที่จะปล่อยหน้าจอโล่ง ๆ

6.2 ฟอร์มเพิ่มงานด้วย POST

ฟอร์มมีช่องชื่องาน หมวดหมู่ และตัวเลือกความสำคัญ เมื่อกด "เพิ่มงาน" จะป้องกันการโหลดหน้าใหม่ด้วย `preventDefault()` แล้วส่ง `fetch` แบบ POST พร้อมหัว `Content-Type: application/json` เมื่อสำเร็จก็ล้างฟอร์มและเรียก `loadTasks()` ใหม่ รายการจึงอัปเดตทันที ถ้าเซิร์ฟเวอร์ตอบ 400 (เช่น ชื่องานเป็นช่องว่าง) จะแสดงข้อความสีแดงใต้ฟอร์ม

6.3 ปุ่มติ๊กเสร็จ ปุ่มแก้ไข และปุ่มลบ

- ติ๊กเสร็จ: checkbox หน้างานเรียก PATCH ส่ง `{"done": true/false}` ชื่องานที่เสร็จแล้วจะมีเส้นขีดทับและสีจางลง
- แก้ไข: กดแล้วแถวนั้นเปลี่ยนเป็นช่องกรอก (ชื่อ หมวดหมู่ ความสำคัญ) กด "บันทึก" จะเรียก PATCH และกด "ยกเลิก" จะกลับสู่โหมดปกติ
- ลบ: กดแล้วมีหน้าต่างยืนยันก่อน จากนั้นเรียก DELETE เมื่อสำเร็จก็โหลดรายการใหม่

6.4 ตัวกรองรายการ

มีแท็บ "ทั้งหมด / ค้างอยู่ / เสร็จแล้ว" และ dropdown เลือกหมวดหมู่ เมื่อเลือกแล้วหน้าเว็บจะประกอบ query string ด้วย `URLSearchParams` แล้วเรียก `GET /api/tasks?done=...&category=...` ดังนั้นการกรองทำที่ฝั่งเซิร์ฟเวอร์จริง ๆ ไม่ใช่ซ่อนแถวด้วย CSS ตัวเลือกใน dropdown สร้างจากหมวดหมู่ที่มีอยู่ในข้อมูล และมีสรุปด้านบนว่า "ค้างอยู่ X จาก Y งาน"

6.5 การออกแบบหน้าตา (UI/UX)

- ธีมสี: พื้นหลังฟ้าเทาอ่อน กระดาษสีขาวตรงกลาง ตัวอักษรน้ำเงินเข้ม และใช้สีน้ำเงินเป็นสีปุ่มหลัก
- บอกความสำคัญด้วยแถบสีซ้ายมือ: ส้มแดง = ด่วน, เหลืองอำพัน = ปกติ, ฟ้าเทา = ไม่รีบ ดูรายการแวบเดียวก็รู้ว่าอะไรต้องรีบทำ
- Responsive: บนจอแคบ (ต่ำกว่า 560px) ฟอร์มและปุ่มจัดเรียงใหม่เป็นแนวตั้ง ใช้งานบนมือถือได้
- การเข้าถึง: ช่องกรอกและ checkbox มี `aria-label` มีกรอบโฟกัสเวลากดแท็บ และเคารพการตั้งค่าลดแอนิเมชัน
- ข้อความ: ใช้คำสั้น ๆ ที่บอกสิ่งที่จะเกิดขึ้น เช่น "เพิ่มงาน" "บันทึก" "ลบ"

6.6 เรียก API ของตัวเองเท่านั้น

`fetch()` ทุกตัวใน `app.js` ชี้ไปที่ `/api/tasks` ของเซิร์ฟเวอร์ตัวเองเท่านั้น ไม่ได้ดึงข้อมูลจาก API ภายนอก มีเพียงฟอนต์ภาษาไทย (Noto Sans Thai) ที่โหลดผ่าน Google Fonts เพื่อความสวยงาม ซึ่งเป็นแค่ไฟล์ฟอนต์ ไม่ใช่ข้อมูลของแอป

7. การทดสอบและภาพหน้าจอการทำงาน

7.1 วิธีทดสอบ

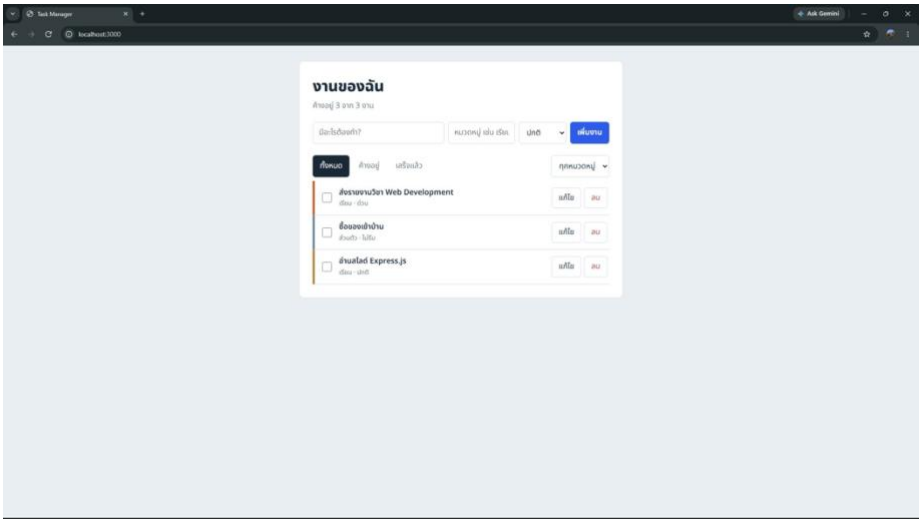
ทดสอบสองทาง ทางแรกคือใช้งานผ่านหน้าเว็บจริง แล้วเปิดแท็บ Network ใน DevTools ดู request และ status code ทางที่สองคือยิง API ตรง ๆ ด้วย curl หรือ Postman เพื่อทดสอบกรณีผิดปกติที่หน้าเว็บไม่ได้ส่ง เช่น ส่งงานที่ไม่มีชื่อ

```
curl http://localhost:3000/api/tasks?done=false
curl -i http://localhost:3000/api/tasks/999
curl -i -X POST http://localhost:3000/api/tasks -H "Content-Type: application/json" -d "{\"title\":\"\"}"
curl -X PATCH http://localhost:3000/api/tasks/1 -H "Content-Type: application/json" -d "{\"done\":true}"
curl -i -X DELETE http://localhost:3000/api/tasks/1
```

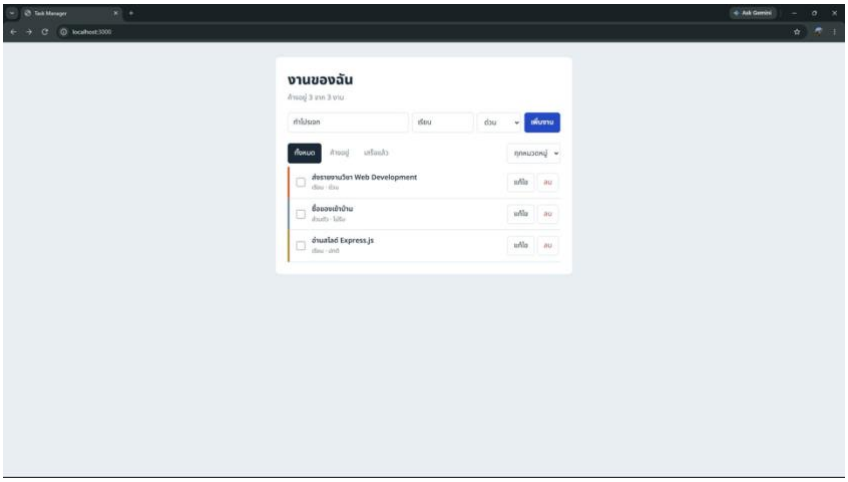
7.2 ตารางกรณีทดสอบ

#	กรณีทดสอบ	ผลที่คาดหวัง	ผลที่ได้
1	GET /api/tasks	200 และอาร์เรย์ของงานทั้งหมด	[ ]
2	GET /api/tasks?done=true	200 เฉพาะงานที่เสร็จแล้ว	[ ]
3	GET /api/tasks/1	200 และงานที่ id 1	[ ]
4	GET /api/tasks/999	404 พร้อมข้อความ error	[ ]
5	POST ข้อมูลครบ	201 และงานใหม่ที่มี id	[ ]
6	POST โดยไม่มี title	400 พร้อมข้อความ error	[ ]
7	PATCH {"done": true}	200 และ done เป็น true	[ ]
8	PATCH id ที่ไม่มี	404	[ ]
9	DELETE id ที่มี	204 ไม่มี body	[ ]
10	DELETE id ที่ไม่มี	404	[ ]

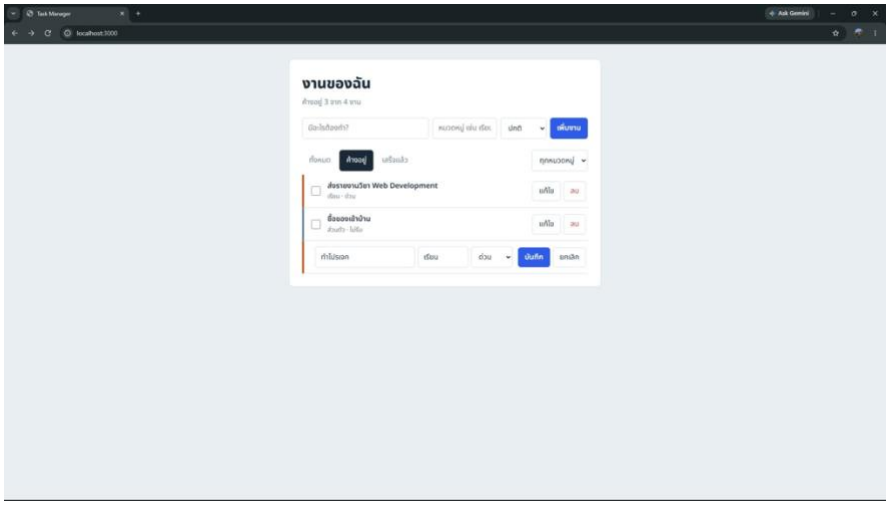
7.3 ภาพหน้าจอการทำงานของเว็บไซต์



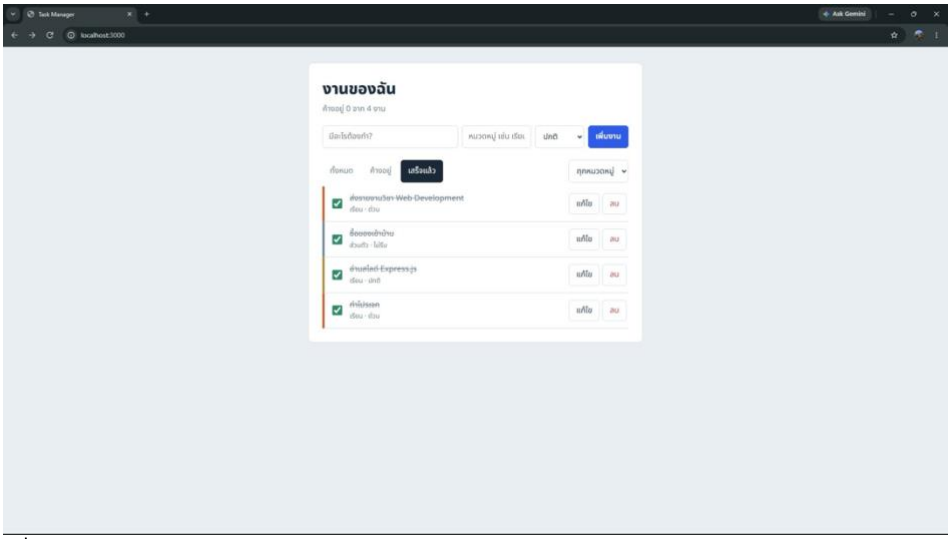
ภาพที่ 1 หน้าหลักแสดงรายการงานที่ดึงมาจาก GET /api/tasks



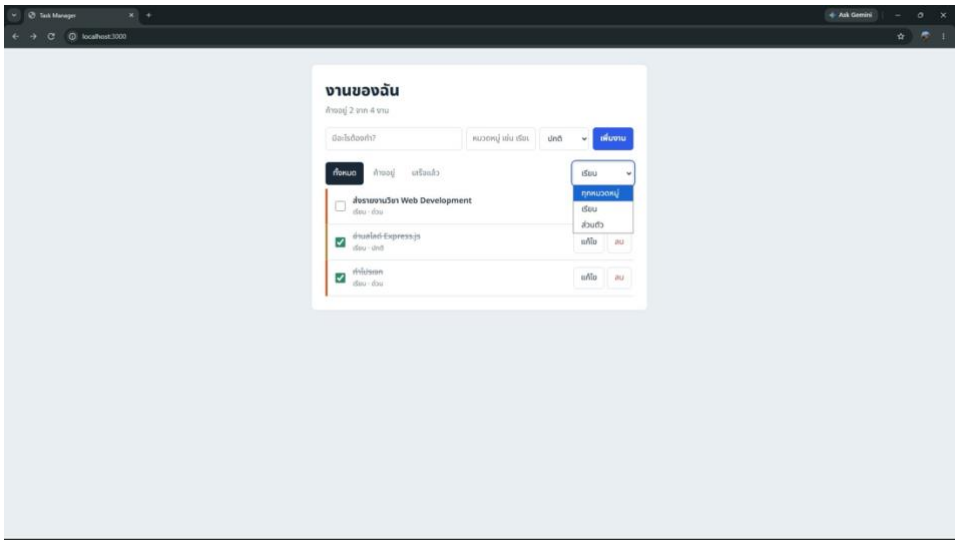
ภาพที่ 2 เพิ่มงานใหม่ด้วยฟอร์ม (POST) รายการอัปเดตโดยไม่รีเฟรช



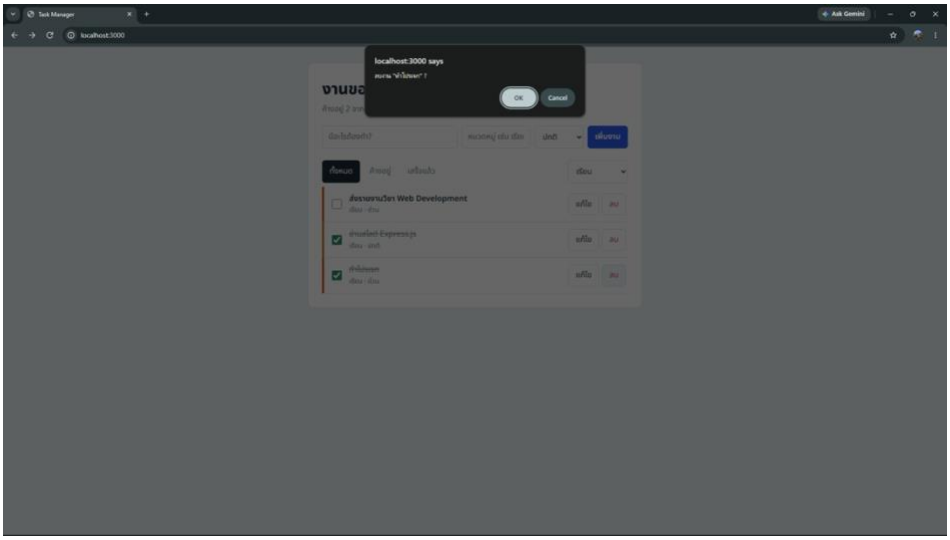
ภาพที่ 3 แก้ไขงานในแถว (PATCH)



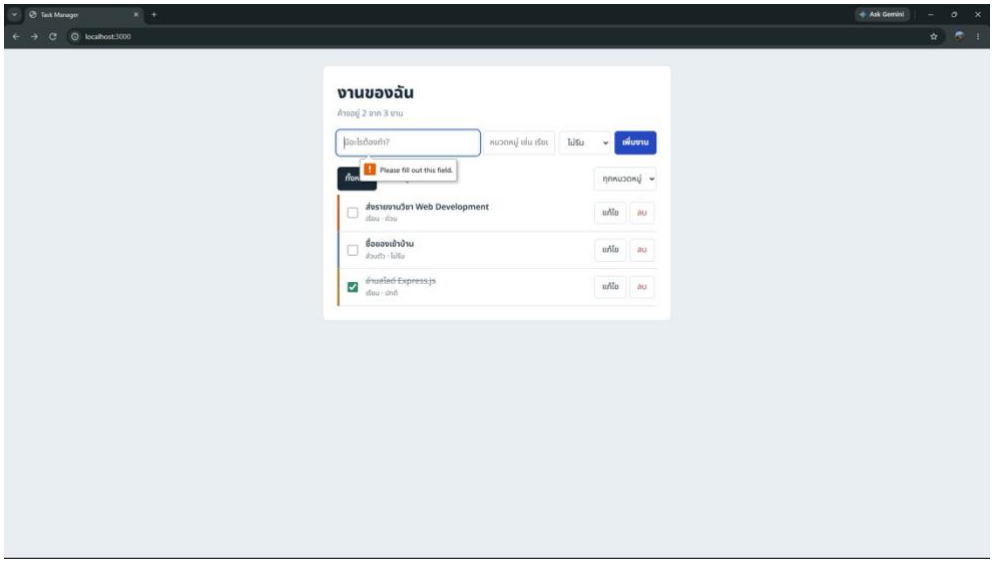
ภาพที่ 4 ติ๊กงานเสร็จและกรองด้วยแท็บสถานะ (?done=true)



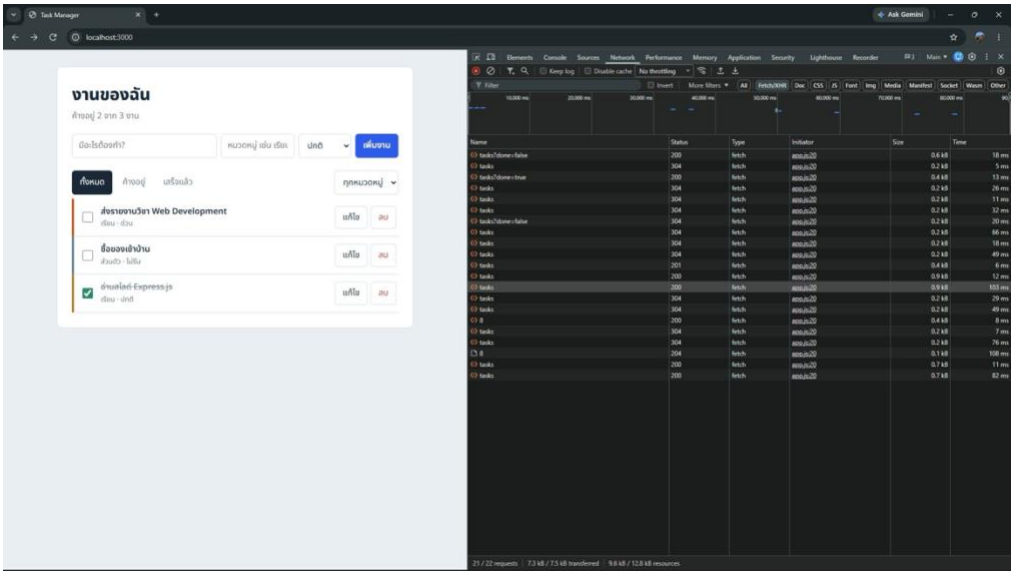
ภาพที่ 5 กรองตามหมวดหมู่ (?category=...)



ภาพที่ 6 ลบงาน (DELETE)



ภาพที่ 7 กรณีข้อมูลไม่ครบ ได้ข้อความ error (400)



ภาพที่ 8 ตรวจ status code จาก DevTools หรือ Postman



ภาพที่ 9 การแสดงผลบนหน้าจอขนาดมือถือ

8. ปัญหาที่ต้องระวังและวิธีจัดการ

8.1 id ใน URL เป็นสตริง

ค่า `req.params.id` เป็นสตริงเสมอ ถ้าเทียบกับ id ที่เป็นตัวเลขตรง ๆ ด้วย `===` จะหาไม่เจอทุกครั้ง จึงต้องแปลงด้วย `Number()` ก่อน ถ้าผู้ใช้ส่ง id เป็นตัวอักษรก็จะได้ `NaN` ซึ่งไม่ตรงกับงานใดเลย ผลคือตอบ 404 ตามที่ควร

8.2 Response แบบ 204 ไม่มี body

การลบสำเร็จตอบ 204 โดยไม่มีเนื้อหา ถ้าฝั่งไคลเอนต์เรียก `res.json()` ต่อจะเกิด error จึงเขียนฟังก์ชัน `request()` ให้เช็ก status 204 แล้วคืนค่า `null` แทน

8.3 ความปลอดภัยเวลาแสดงข้อความของผู้ใช้

ชื่องานเป็นข้อความที่ผู้ใช้พิมพ์เอง ถ้าเอาไปต่อเป็น HTML ด้วย `innerHTML` อาจมีคนใส่แท็ก `<script>` ได้ (XSS) เราจึงใช้ `textContent` ซึ่งแสดงเป็นข้อความธรรมดาเสมอ

8.4 JSON เสียรูปแบบและ Content-Type

ถ้าลืมใส่หัว `Content-Type: application/json` ตอน POST เซิร์ฟเวอร์จะอ่าน body ไม่ได้ และ `req.body` จะไม่มีข้อมูล จึงต้องใส่หัวนี้ทุกครั้งใน `fetch()` และฝั่งเซิร์ฟเวอร์มีตัวดักกรณี JSON ผิดรูปแบบให้ตอบ 400 ด้วย

8.5 การตรวจข้อมูลต้องทำที่เซิร์ฟเวอร์

ช่องชื่องานในหน้าเว็บมี `required` อยู่แล้ว แต่ผู้ใช้ข้ามหน้าเว็บไปยิง API ตรง ๆ ได้ จึงต้องตรวจที่เซิร์ฟเวอร์เสมอ การตรวจฝั่งไคลเอนต์ช่วยแค่ให้ใช้งานสะดวกขึ้น

9. สรุปผลและแนวทางพัฒนาต่อ

9.1 สรุปผลการทำงาน

โปรเจกต์นี้ได้ REST API ที่ครบ CRUD ทั้ง 5 เส้นทาง พร้อมการกรองด้วย query string การตรวจข้อมูล และ status code ที่ตรงตามโจทย์ (200, 201, 204, 400, 404) และมีหน้าเว็บที่เรียก API ของตัวเองด้วย `fetch()` เพิ่ม แก้ไข ลบ และกรองงานได้โดยไม่ต้องโหลดหน้าใหม่ สิ่งที่ได้เรียนรู้หลัก ๆ คือเข้าใจว่า HTTP method กับ status code แต่ละตัวมีความหมายอะไร และรู้ว่าการแยกหน้าที่ระหว่างฝั่งเซิร์ฟเวอร์กับไคลเอนต์ช่วยให้โค้ดเป็นระเบียบขึ้นมาก

9.2 ข้อจำกัด

- ข้อมูลเก็บในหน่วยความจำ จึงหายเมื่อรีสตาร์ทเซิร์ฟเวอร์
- ไม่มีระบบล็อกอิน ทุกคนที่เข้าเซิร์ฟเวอร์เดียวกันเห็นข้อมูลชุดเดียวกัน
- ยังไม่มีการแบ่งหน้า (pagination) ถ้างานเยอะมากรายการจะยาว
- ไม่ได้ host ให้ใช้งานบนอินเทอร์เน็ต

9.3 แนวทางพัฒนาต่อ

- เปลี่ยนไปเก็บข้อมูลลงไฟล์ JSON หรือฐานข้อมูล เช่น SQLite หรือ MongoDB
- เพิ่มวันกำหนดส่ง (due date) และเรียงงานตามวันที่
- เพิ่มช่องค้นหาชื่องาน และ pagination
- เพิ่มระบบผู้ใช้ เพื่อให้แต่ละคนมีรายการงานของตัวเอง

ภาคผนวก: วิธีติดตั้งและรัน

```
git clone https://github.com/Mosfox/miniProject
cd task-manager
npm install
npm run dev
```

จากนั้นเปิดเบราว์เซอร์ไปที่ `http://localhost:3000` รายละเอียด endpoint ทั้งหมดอยู่ในไฟล์ `README.md`